

CockroachDB Distributed SQL

L0 Essence: 分布式 SQL 数据库的本质是利用 Multi-Raft 进行水平 Range 分片共识复制，基于混合逻辑时钟（HLC）和 MVCC 实现分布式事务一致性与快照隔离，使得无共享架构集群呈现单机关系型 SQL 数据库的体验。

[M1.1] SQL 解析与分发执行：Gateway 节点的两阶段分发

Gateway 节点解析 SQL 生成抽象语法树（AST），经语义分析后重写为逻辑计划。优化器识别查询涉及的 KV 范围，通过 Meta 索引树定位 Range 所在的物理节点，将逻辑计划切分为多个物理分片计划投递到各节点并行执行。

[M1.2] 分布式 SQL 执行引擎（DistSQL）流式处理与 Flow 拓扑

DistSQL 引擎将分布式物理计划转换为 Flow 拓扑。Flow 包含多个 Processor 算子（如 TableReader, Joiner）和通过网络连接的 Routers。元组（Tuple）在算子流式管道中以批（Batch）形式 Push/Pull 传输，极大减小了 Gateway 节点的中间汇总带宽负荷。

[M1.3] 分布式 Scan 算子与主键/二级索引 KV 编解码规则

CockroachDB 将关系数据表映射为扁平 KV 空间。主键编码规则为：Prefix/TableID/IndexID/PrimaryKey → ColumnValues；二级索引编码规则为：Prefix/TableID/IndexID/SecondaryKey/PrimaryKey → void。Scan 算子基于前缀匹配直接进行多区间快速检索。

[M1.4] 优化器基于代价的路径选择（CBO）与统计信息维护

优化器基于 Cascades 框架计算物理计划的代价。系统通过收集等深直方图、多列关联性分布以及物理跨机房延迟参数，在 CBO 计算中评估多种 Join 算法（Lookup Join, Merge Join, Hash Join），以选择网络传输代价最低的执行拓扑。

[M2.1] Range 动态分片分裂（Range Splits）与合并机制

表数据按主键顺序划分为连续 Range，默认上限为 64MB。当持续写入使得 Range 空间超限，或者检测到单一 Range 负载过高时，节点在本地启动 Split 流程，切分数据并向全局三级 Meta 索引树注册新子 Range。相反，当数据删除造成 Range 过小时，触发合并（Merge）。

[M2.2] Multi-Raft 共识架构：多 Raft Group 隔离与心跳合并

每个 Range 对应一个独立的 Raft Group。单节点可能同时托管数万个副本，传统心跳会引发系统级“心跳风暴”。Multi-Raft 引擎将发送到同一物理节点的多个 Raft 实例心跳打包合并（Coalesced Heartbeats），减少了网络 IO 与 CPU 中断消耗。

[M2.3] 副本租约（Leaseholder）机制：读写流向与脑裂防范

为避免 Raft 多数派读的网络开销，每个 Range 选举产生一个 Leaseholder（通常与 Raft Leader 重合）。所有读写请求必须路由至 Leaseholder。Leaseholder 本地读能确保线性一致性，写请求则由 Leaseholder 发起 Raft 复制提案，以此彻底规避了脑裂并发。

[M2.4] 副本重平衡（Rebalancing）与 Raft 成员变更控制

Allocator 周期性收集各节点资源指标。当检测到节点间磁盘水位差或 CPU 负载失衡时，Allocator 基于物理拓扑计算最佳目标节点，使用 Raft Joint Consensus 联合共识的两阶段机制迁移 Range 副本，迁移期间对前台读写无中断感知。

[M3.1] 分布式两阶段提交（2PC）原子性与事务记录（Transaction Record）

CockroachDB 分布式事务基于 2PC 实现。事务在系统 Range 创建一条 Transaction Record，初始状态为 PENDING。一阶段对所有涉及分片写入数据；二阶段仅将该 Record 状态原子修改为 COMMITTED，即便后续清理中断，也可依据此状态判定全局可见性。

[M3.2] 写意图（Write Intent）锁机制与行级并发冲突检测

事务写数据时不直接覆盖旧值，而是生成 Write Intent——一个包含新值与指向 Transaction Record 的 MVCC 临时记录，等同于行锁。其他事务读取时，若读到 Intent，会根据指针追溯原事务的 Transaction Record 状态，从而判定该 Intent 是否已提交生效。

[M3.3] 两阶段提交的优化：单阶段提交（1PC）与非冲突并发

如果事务的所有写入键全部位于同一个 Range 内，协调节点将跳过分布式 2PC。事务日志与提交指令直接合并为单条 Raft 日志在当前 Multi-Raft Group 中复制并应用。该优化（1PC）将分布式事务的网络延迟压低至单次本地共识开销。

[M3.4] 事务清理（Intent Cleanup）与异步垃圾回收

一旦 Transaction Record 的状态被更新为 COMMITTED 或 ABORTED，事务协调器会触发异步清理（Intent Cleanup）任务，将临时写意图 Write Intent 转化为真正的 MVCC 历史版本（或直接清除以解冻行锁），该过程不阻塞后续已就绪的读写事务。

[M3.5] 读写冲突解决机制：事务优先级与推迟重试

当事务遭遇其他未提交的 Write Intent 时，会引发读写/写写冲突。系统通过计算事务优先级（Priority，融合重试次数与时延）决定抢占规则。高优先级事务可以直接 Push 对方事务状态（促使其 Abort），低优先级事务则在本地排队等待或回滚重启。

L1 Logical Framework

- 数据分治与共识分发:** 将表自动划分为 64MB 的 Range 分片，每个 Range 绑定 Multi-Raft Group 并依靠租约（Leaseholder）拦截读写。
- 分布式事务与锁意图:** 通过二阶段提交（2PC）修改单条原子事务记录，配合行级写意图（Write Intent）锁实现全局 ACID。
- 混合时钟与冲突自愈:** 利用 HLC 逻辑推进与不确定性区间等待协议，破除物理时钟漂移，实现快照隔离与线性一致性。
- 动态扩容与在线升级:** 通过 Gossip 成员发现与代价值 Allocator 动态搬迁 Range，实现零停机的物理容灾与非阻塞 Schema 变更。

L2 Data Flow Topology

Client SQL	Gateway Parse	Lookup Route Range	Locate Leaseholder
Raft Command	Write Intent	Create PENDING Transaction Record	Raft
Consensus Commit	Update Transaction Record to COMMITTED	Async Intent	
Cleanup	Return OK.		

Trade-off Matrix: Concurrency & Availability

\textbf{设计维度 (Design Dimension): 方案 A (Approach A) → 方案 B (Approach B) [折衷折度 (Trade-off Balance)]
\textbf{一致性级别 vs 读写时延: 强串行化隔离级别 (SSI) → 弱隔离级别 (SI / RC) [SSI 提供完全无并发异常的安全保证 → 但在高冲突写入时会导致频繁的 Read Restart 事务重启与读延迟 [时延升高, 吞吐受限]]
\textbf{时钟源精度 vs 脑裂防范: HLC 混合逻辑时钟 (物理 + 逻辑) → 严格硬件 TrueTime 物理时钟 (原子钟 + GPS) [HLC 摆脱了昂贵的硬件依赖, 具备极强普适性 → 但必须设置 Clock Offset 不确定度等待, 时钟偏差过大时必须主动 Panic 熔断 [牺牲节点局部可用性, 确保一致]]
\textbf{分布式事务 vs 冲突窗口: 2PC 协调两阶段提交 + 写意图 → 1PC 单阶段提交优化 [2PC 支持跨 Range 任意原子交易 → 但 1PC 只能应用于单分片 Range 写事务。通过限制使用范围消除 2PC 的两阶段通信延迟 [降低冲突窗口, 提升小事务性能]]
\textbf{复制组分片 vs 拓扑心跳数: Coalesced Heartbeats 合并心跳 → 单副本组单独心跳探测 [合并心跳极大地降低了单节点万级 Range 副本下的网络风暴与 CPU 中断 → 但由于合并发送, 单个 Range 状态的即时刷新与探测敏感度有所降低 [降低资源开销, 牺牲探测时效]]

[M4.1] 混合逻辑时钟（HLC）物理与逻辑复合公式

HLC 替代了昂贵的 TrueTime 硬件。HLC 时间戳由物理读数 $l.i$ 和逻辑计数 $c.i$ 组合表示。如果本地物理时钟增量大于已知的最大集群时钟值，则推进物理时间；如果物理时间未变但发生因果事件，则累加 $c.i$ 计数，保证强因果偏序关系。

[M4.2] 时钟偏移不确定性（Clock Offset Uncertainty）与不确定度读取重试

在物理时钟存在漂移的最大偏差（ max_offset ）范围内，若读快照时间戳 T_{read} 落在可能导致时序颠倒的“不确定性区间” $[T_{read}, T_{read} + \text{max_offset}]$ 内，读事务必须执行 Read Restart，更新自身读取时间戳重试，以防御读写偏斜。

[M4.3] HLC 逻辑时钟的 Tick 与 Update 事件推进协议

- **Tick (本地事件):** $l_i = \max(l_i, \text{physical_time})$ 。若物理时间增加，则重置 $c_i = 0$ ，否则 $c_i = c_i + 1$ 。
- **Update (收发消息):** 接收包含 $T_{msg} = (l_{msg}, c_{msg})$ 的 RPC 时，更新 $l_i = \max(l_i, l_{msg}, \text{physical_time})$ ，并据此决定重置逻辑计数或合并递增。

[M4.4] 跨地域只读副本（Follower Reads）与 HLC 时间戳对齐

Follower Reads 允许只读事务避开 Leaseholder。客户端发起读取时，指定一个安全的历史时间戳 $T_{read} \leq HLC_{local} - \text{max_offset}$ 。由于该时间戳已超出所有可能并发修改的偏差区间，Follower 副本可以直接本地提供强一致读。

[M5.1] MVCC 物理存储布局与多版本 Key 编解码

底层物理引擎将多版本行数据编码存储。写入同一个 Key 时，在底层持久化为一系列带有 HLC 时间戳后缀的独立键值对：Key/Timestamp $\rightarrow$ Value。新写入的时间戳最高，读操作带快照时间戳，根据前缀 Scan 查找并返回时间戳小于等于该快照的最接近版本。

[M5.2] 读屏障（Read Barrier）与快照隔离级别（SI）实现

只读事务获取当前的 HLC 作为 T_{read} ，以此作为可见性读屏障。读取过程中，若遭遇时间戳 $\leq T_{read}$ 的 Write Intent，事务必须等待锁持有者提交或回滚；若遭遇时间戳 $> T_{read}$ 的新版本或锁，读屏障直接过滤，提供 SI 级别隔离。

[M5.3] 串行化隔离级别（SSI）与读写冲突依赖环防范

CockroachDB 仅支持最高级别的 Serializable 隔离。系统利用 Read Timestamp Cache 追踪每个键被读取的最大时间戳。如果有写事务试图写入已被高时间戳读过的 Key，系统利用此 Cache 自动检测到反向依赖（rw-antidependency），强制写事务回滚重启。

[M5.4] MVCC 历史版本物理回收（GC TTL）与 Compaction 清理

每个 Zone 配置了 MVCC 回收期限（GC TTL）。后台 GC 队列持续扫描各个 Range，将超期的旧版本数据标记为“已物理删除”。底层存储引擎（Pebble）在执行后台 LSM-Tree 压实（Compaction）时，彻底擦除这些废弃键值，重构 SSTable 释放磁盘。

[M6.1] Gossip 协议集群成员拓扑自动发现与状态广播

集群节点启动时，向配置好的 Seed 节点发送握手请求。通过 Gossip 协议，所有节点以完全去中心化的 Peer-to-Peer 方式异步广播各自的在线状态、物理拓扑标签（Locality）、磁盘水位分布以及最新的网络延迟，达成全局拓扑一致。

[M6.2] Node/Rack/Datacenter 物理感知与容灾放置策略

副本分配器（Allocator）严格遵循 Zone 配置的 Locality 约束（机房、机架标签）。在进行副本分配时，Allocator 会将 Raft 多数派（例如 3 副本中的 2 个）强制打散到不同的故障域（Rack/Datacenter），保证节点级和区域级断电时的服务活性。

[M6.3] 动态数据迁移与副本数平滑在线调整

用户在线修改表的复制策略时，Allocator 会计算扩缩容步骤。通过 Raft 成员变更机制向目标节点派发副本，以 Raft Learner 模式同步追平 WAL 历史日志后激活为正式 Follower，最后平滑剥离并销毁被替代的旧副本，全局无感知。

[M6.4] 物理 Schema 变更：非阻塞表结构在线升级

在线添加二级索引或新增非空列时，CockroachDB 将结构变更划分为多个过渡版本：Delete-only（只删不插） $\rightarrow$ Write-only（只写不读） $\rightarrow$ Public（读写皆可）。在后台逐步回填历史数据，避免了传统大表长时间锁表带来的写中断。

[M6.5] 慢节点（Stuck Nodes）隔离与 Lease 租约自动流转

当某个 Leaseholder 因为磁盘 IO 悬挂或网络丢包发生抖动时，其 Lease Heartbeat 心跳会超时。该 Range 的其他 Follower 副本在检测到此情况后，快速发起 Lease 抢占请求，将读写流转到健康节点，而无需等待该宕机节点重启。

[M6.6] Hotspot 热点分片（Hotspot）自动散列分发与限流保护

Range 写入吞吐量（QPS）超过设定限额时，判定为 Hotspot。Allocator 会立刻下发 Range 分裂指令（Hot Split），并强行将分裂后的新 Leaseholder 迁移调度到其他空闲 CPU 的物理节点上，配合客户端路由由热更分散突发并发流量。

[M6.7] 时间同步心跳探测与时钟崩溃自动熔断隔离

每个物理节点运行有 NTP 监控心跳。当节点检测到自身物理时间与集群多数派节点的 HLC 时钟偏差超过 server.clock.max_offset 阈值（默认 200ms）时，会立即触发自我隔离（Auto-Panic 熔断），以防止时钟失控写入脏数据破坏强一致性。

Zone T1: CockroachDB Core Parameters

kv.range.max_bytes = 64MB ; Range 触发 Split 动态分裂的最大物理数据大小

限制。server.clock.max_offset = 200ms : 允许的集群节点间物理时钟最大漂移上限。超出此值节点自动 Panic 隔离。kv.transaction.max_intents_bytes = 256KB : 单事务内存中 Write Intent 临时写意图锁缓存上限，超限落盘以防 OOM。

sql.defaults.vectorization.enabled = true: 默认开启 DistSQL 的物理算子 CPU SIMD 向量化计算加速。kv allocator.range_rebalance_threshold = 0.05 : Range 副本重平衡分配的节点间存储容量差阈值（5%）。

Zone T2: Diagnostics & Troubleshooting

cockroach debug zip : 一键打包收集全集群中所有节点的日志、性能指标、配置与运行时堆栈，用于离线故障诊断。cockroach node status --ranges : 查看各个物理节点上 Range 的持有分布、Leaseholder 分布及落后复制副本的状态。cockroach sql -e "SHOW CLUSTER SETTINGS" : 检查集群当前所有的全局参数限制（时钟最大偏移、分裂大小等）。cockroach debug range-log <range-id> : 检索指定 Range 分片底层的 Raft 复制日志流、租约流转与分裂合并的历史变更。chronyc tracking/ntpstat : 在操作系统物理侧实时校验本地时钟同步误差，预防逼近 200ms 熔断阈值。